

ui.codemirror 全面详细阐述

`ui.codemirror` 是 NiceGUI 框架中基于 **CodeMirror 6** 实现的高级代码编辑器组件，支持 100+ 编程语言语法高亮、代码折叠、主题切换、行号显示、自动补全、快捷键绑定等专业功能，适用于代码编辑、配置文件编写、代码演示等开发场景。以下从核心特性、使用方法、参数配置、API 详情等方面展开全面说明。

一、核心特性

- 1. **强大语法支持**：内置 100+ 编程语言语法高亮（如 Python、JavaScript、HTML、JSON、YAML 等），支持自定义语法配置。
- 2. **丰富编辑功能**：支持行号显示、代码折叠、缩进控制、自动补全、括号匹配、拼写检查（需插件）。
- 3. **主题高度自定义**：内置 20+ 预设主题（亮色 / 暗色），支持自定义 CSS 主题，适配不同界面风格。
- 4. **灵活事件监听**：支持内容变化、编辑器聚焦 / 失焦、光标移动等事件回调，可实时响应编辑操作。
- 5. **配置深度可调**：支持通过 `options` 参数传递 CodeMirror 原生配置（如只读模式、缩进大小、换行方式），支持插件扩展（如代码格式化、LSP 集成）。
- 6. **组件联动能力**：支持与其他组件（如下拉框、开关）绑定，实现主题切换、语言切换、内容同步等交互逻辑。
- 7. **版本兼容特性**：NiceGUI 2.0.0+ 引入，3.0.0+ 支持更多 CodeMirror 6 原生配置，需注意框架版本与 CodeMirror 插件兼容性。

二、基础使用方法

1. 最简示例：Python 代码编辑器

快速创建支持 Python 语法高亮的编辑器，实时监听内容变化并显示：

```
from nicegui import ui

# 标签用于实时显示编辑器内容
code_display = ui.label('编辑器内容将显示在这里...').classes('whitespace-pre-wrap')

# 基础 Python 代码编辑器
editor = ui.codemirror(
    value='print("Hello, CodeMirror!")\nfor i in range(5):\n    print(i)',
    language='python', # 指定编程语言（语法高亮）
    theme='default', # 预设主题（default 为亮色主题）
    on_change=lambda e: code_display.set_text(f'当前代码: \n{e.value}') # 内容变化回调
)

ui.run()
```

2. 核心功能配置（行号、折叠、主题）

通过 `options` 参数配置行号、代码折叠、缩进大小等基础功能，结合下拉框切换主题：

```
from nicegui import ui

# 预设主题列表（CodeMirror 6 内置主题）
```

```

themes = ['default', 'dracula', 'monokai', 'solarized light', 'solarized dark', 'github']

# 主题切换下拉框
theme_select = ui.select(themes, value='default', label='选择主题')

# 高级配置编辑器
editor = ui.codemirror(
    value='''def add(a, b):
    """简单加法函数"""
    return a + b

# 代码折叠示例
if __name__ == "__main__":
    result = add(10, 20)
    print(f"结果: {result}")'''
    language='python',
    theme=theme_select.value,
    # CodeMirror 原生配置（完整参数见 CodeMirror 6 文档）
    options={
        'lineNumbers': True,          # 显示行号
        'foldGutter': True,          # 启用代码折叠（侧边 gutter）
        'gutters': ['CodeMirror-linenumbers', 'CodeMirror-foldgutter'], # 显示行号和折叠
    gutter
        'indentUnit': 4,             # 缩进大小（4 个空格）
        'tabSize': 4,                 # Tab 键等效空格数
        'lineWrapping': True,         # 自动换行
        'readOnly': False,            # 可编辑模式（True 为只读）
    }
)

# 主题切换联动：下拉框变化 → 编辑器主题同步更新
theme_select.bind_value_to(editor, 'theme')

ui.run()

```

3. 多语言语法高亮切换

通过下拉框切换编程语言，实现不同文件类型的语法高亮适配：

```

from nicegui import ui

# 支持的编程语言列表（CodeMirror 6 内置语言）
languages = [
    ('Python', 'python'),
    ('JavaScript', 'javascript'),
    ('HTML', 'html'),
    ('CSS', 'css'),
    ('JSON', 'json'),
    ('YAML', 'yaml'),
    ('SQL', 'sql'),
]

```

```

# 语言切换下拉框
lang_select = ui.select(
    [lang[0] for lang in languages],
    value='Python',
    label='选择编程语言'
)

# 编辑器初始配置
editor = ui.codemirror(
    value='print("Python 代码")',
    language='python',
    theme='monokai',
    options={'lineNumbers': True}
)

# 语言切换逻辑：下拉框选择 → 更新编辑器语言和默认代码
def switch_language():
    selected_lang_name = lang_select.value
    selected_lang_code = next(lang[1] for lang in languages if lang[0] ==
selected_lang_name)
    # 更新编辑器语言
    editor.language = selected_lang_code
    # 根据语言设置默认代码
    default_code = {
        'python': 'print("Python 代码")',
        'javascript': 'console.log("JavaScript 代码");',
        'html': '<!DOCTYPE html>\n<html>\n  <body>\n    <h1>HTML 代码</h1>\n
</body>\n</html>',
        'css': 'body {\n  color: red;\n}',
        'json': '{\n  "name": "CodeMirror",\n  "type": "editor"\n}',
        'yaml': 'name: CodeMirror\ntype: editor',
        'sql': 'SELECT * FROM users;',
    }[selected_lang_code]
    editor.set_value(default_code)

# 绑定下拉框变化事件
lang_select.on_change(lambda _: switch_language())

ui.run()

```

4. 只读模式与代码预览

通过 `options={'readOnly': True}` 启用只读模式，适用于代码演示、配置文件预览等场景：

```

from nicegui import ui

# 只读模式编辑器（代码预览）
ui.codemirror(
    value=''# 只读模式示例
# 此编辑器仅用于预览，不可编辑
def demo():
    return "CodeMirror 只读模式"

```

```
language='python',
theme='github',
options={
    'lineNumbers': True,
    'readOnly': True, # 启用只读模式
    'lineWrapping': True,
}
).classes('h-64') # 设置编辑器高度 (Tailwind 类)

ui.run()
```

5. 事件监听（聚焦、失焦、内容变化）

监听编辑器核心事件，实现交互反馈（如聚焦时高亮边框、失焦时保存内容）：

```
from nicegui import ui

editor = ui.codemirror(
    value='# 编辑代码...',
    language='python',
    theme='dracula',
    options={'lineNumbers': True}
).classes('h-64 border-2 border-gray-300 transition-all')

# 聚焦事件：边框变色
editor.on('focus', lambda _: editor.classes('border-blue-500'))
# 失焦事件：边框恢复默认色 + 保存内容
def on_blur(_):
    editor.classes('border-gray-300')
    ui.notify(f'代码已保存（长度：{len(editor.value)} 字符）')

editor.on('blur', on_blur)
# 内容变化事件：实时统计字符数
char_count = ui.label(f'字符数：{len(editor.value)}')
editor.on_change(lambda e: char_count.set_text(f'字符数：{len(e.value)}'))

ui.run()
```

三、关键参数说明

参数名	类型	说明	默认值	
value	<code>str</code>	None	编辑器初始内容，支持多行文本（含换行符 <code>\n</code> ）	None
language	<code>str</code>	编程语言标识（如 <code>python</code> 、 <code>javascript</code> ），决定语法高亮规则，需为 CodeMirror 6 支持的语言	<code>'text'</code> （纯文本）	

参数名	类型	说明	默认值	
theme	str	编辑器主题，支持 CodeMirror 6 内置主题（如 default、dracula）或自定义主题	'default'	
options	`dict	None`	CodeMirror 6 原生配置字典，支持所有 CodeMirror 核心配置（如行号、折叠、只读等）	None (默认配置)
on_change	`Callable[[ValueChangeEventArguments], Any]	Callable[[], Any]`	内容变化时触发的回调函数，事件对象 e 含 value 属性（当前编辑器内容）	None
classes	str	组件 HTML 类名，支持 Tailwind/Quasar 类（如设置高度、边框、阴影）	''	
style	str	组件内联 CSS 样式（如 height: 400px;）	''	
html_id	`str	None`	组件 HTML DOM ID（版本 2.16.0+），用于手动操作 DOM 元素	None

核心 options 配置详解（CodeMirror 原生）

options 键名	类型	说明	
lineNumbers	bool	是否显示行号（默认 False）	
foldGutter	bool	是否启用代码折叠（需配合 gutters 配置，默认 False）	
gutters	list	显示的侧边 gutter 列表（如 ['CodeMirror-linenumbers', 'CodeMirror-foldgutter']）	
indentUnit	int	缩进大小（单位：空格，默认 2）	
tabSize	int	Tab 键等效空格数（默认 2）	
lineWrapping	bool	是否自动换行（默认 False）	
readOnly	`bool	str`	是否只读：True（完全只读）、'nocursor'（无光标只读）、False（可编辑）
matchBrackets	bool	是否自动匹配括号（()、[]、{}，默认 False）	

options 键名	类型	说明	
autoCloseBrackets	bool	是否自动闭合括号（默认 False）	
placeholder	str	编辑器为空时显示的占位提示（默认 ''）	
fontSize	str	字体大小（如 14px、1rem，默认继承父元素）	

四、高级功能

1. 自定义主题（CSS 扩展）

通过 `style` 或自定义 CSS 覆盖主题样式，实现个性化外观：

```
from nicegui import ui

# 自定义主题 CSS（覆盖默认主题颜色）
custom_theme_css = '''
/* 自定义编辑器背景色和文本色 */
.CodeMirror {
    background-color: #f8f9fa !important;
    color: #2d3748 !important;
}
/* 自定义行号颜色 */
.CodeMirror-linenumber {
    color: #718096 !important;
    background-color: #edf2f7 !important;
}
/* 自定义选中内容背景色 */
.CodeMirror-selected {
    background-color: #bee3f8 !important;
}
/* 自定义关键字颜色（Python 示例） */
.cm-keyword { color: #7c3aed !important; }
.cm-string { color: #059669 !important; }
.cm-comment { color: #94a3b8 !important; }
'''

# 注入自定义 CSS
ui.add_css(custom_theme_css)

# 使用自定义主题的编辑器
ui.codemirror(
    value='''# 自定义主题示例
def custom_theme_demo():
    print("自定义关键字、字符串、注释颜色")
    return "自定义主题''',
    language='python',
    theme='default', # 基于默认主题扩展
```

```
        options={
            'lineNumbers': True,
            'matchBrackets': True,
        }
    ).classes('h-64')

ui.run()
```

2. 组件联动：编辑器内容同步到文本框

将 CodeMirror 编辑器与 `ui.textarea` 绑定，实现内容双向同步（适用于代码编辑 + 预览场景）：

```
from nicegui import ui

# 编辑器
editor = ui.codemirror(
    value='print("双向同步示例")',
    language='python',
    theme='monokai',
    options={'lineNumbers': True}
).classes('h-48')

# 分隔线
ui.separator()

# 文本框（与编辑器双向绑定）
textarea = ui.textarea(
    label='同步预览',
    value=editor.value,
    rows=5
)

# 编辑器 → 文本框：实时同步
editor.on_change(lambda e: textarea.set_value(e.value))
# 文本框 → 编辑器：实时同步
textarea.on_change(lambda e: editor.set_value(e.value))

ui.run()
```

3. 代码格式化（结合第三方库）

通过 `black`（Python 格式化）、`prettier`（多语言格式化）等第三方库，实现代码一键格式化：

```
from nicegui import ui
import black # 需安装: pip install black

# Python 编辑器
editor = ui.codemirror(
    value='''def format_me(a,b):
return a+b''',
    language='python',
    theme='github',
```

```

        options={'lineNumbers': True}
    ).classes('h-48')

# 格式化按钮
def format_code():
    try:
        # 使用 black 格式化 python 代码
        formatted_code = black.format_str(
            editor.value,
            mode=black.FileMode(line_length=88)
        )
        editor.set_value(formatted_code)
        ui.notify('代码格式化成功! ')
    except Exception as e:
        ui.notify(f'格式化失败: {str(e)}', type='error')

ui.button('一键格式化代码', on_click=format_code)

ui.run()

```

4. 快捷键绑定 (CodeMirror 原生)

通过 `options` 配置自定义快捷键，实现常用操作（如保存、格式化）的快捷触发：

```

from nicegui import ui

# 编辑器（配置自定义快捷键）
editor = ui.codemirror(
    value='# 快捷键示例: Ctrl+S 保存, Ctrl+F 格式化',
    language='python',
    theme='dracula',
    options={
        'lineNumbers': True,
        # 自定义快捷键 (CodeMirror 6 语法)
        'extraKeys': {
            # Ctrl+S: 保存代码
            'Ctrl-S': lambda view: ui.notify('代码保存成功! '),
            # Ctrl+F: 格式化代码（需结合第三方库，此处仅演示快捷键）
            'Ctrl-F': lambda view: ui.notify('触发格式化（需结合格式化逻辑）'),
            # Ctrl+/: 注释/取消注释（部分语言原生支持）
            'Ctrl-/': lambda view: view.dispatch({
                'changes': view.state.lineBreakBetween(
                    view.state.selection.ranges[0].from_,
                    view.state.selection.ranges[0].to_
                )
            })
            # 注：完整注释逻辑需结合语言特性，可参考 CodeMirror 插件
        }
    }
)

ui.run()

```


5. 动态修改编辑器配置

通过 `set_options()` 方法动态更新 CodeMirror 配置（如切换行号显示、只读模式）：

```
from nicegui import ui

# 编辑器
editor = ui.codemirror(
    value='# 动态配置示例',
    language='python',
    theme='default',
    options={'lineNumbers': True}
).classes('h-48')

# 开关：控制行号显示
line_numbers_switch = ui.switch(label='显示行号', value=True)
line_numbers_switch.on_change(
    lambda e: editor.set_options({'lineNumbers': e.value})
)

# 开关：控制只读模式
read_only_switch = ui.switch(label='只读模式', value=False)
read_only_switch.on_change(
    lambda e: editor.set_options({'readOnly': e.value})
)

# 按钮：切换自动换行
wrap_button = ui.button('切换自动换行', on_click=lambda: editor.set_options(
    {'lineWrapping': not editor.options.get('lineWrapping', False)})
))

ui.run()
```

五、API 详情补充

1. 核心属性

属性名	类型	说明	
<code>value</code>	<code>BindableProperty</code>	编辑器当前内容（可绑定，支持动态同步）	
<code>language</code>	<code>str</code>	当前编程语言（可直接赋值修改，如 <code>editor.language = 'javascript'</code> ）	
<code>theme</code>	<code>str</code>	当前主题（可直接赋值修改，如 <code>editor.theme = 'dracula'</code> ）	
<code>options</code>	<code>dict</code>	CodeMirror 原生配置（可通过 <code>set_options</code> 动态修改）	

属性名	类型	说明	
<code>classes</code>	<code>str</code>	组件 HTML 类名（支持 Tailwind/Quasar 类）	
<code>style</code>	<code>str</code>	组件内联 CSS 样式	
<code>html_id</code>	<code>`str`</code>	<code>None`</code>	组件 HTML DOM ID (版本 2.16.0+)
<code>visible</code>	<code>BindableProperty</code>	组件是否可见（可绑定，支持动态切换）	

2. 常用方法

方法名	说明	参数
<code>set_value(value: str)</code>	动态设置编辑器内容	<code>value</code> : 新代码内容（支持多行文本）
<code>get_value() -> str</code>	获取当前编辑器内容	无参数，返回字符串形式的代码内容
<code>set_options(options: dict)</code>	动态更新 CodeMirror 配置	<code>options</code> : 配置字典（如 <code>{'lineNumbers': True, 'readOnly': False}</code> ）
<code>set_language(language: str)</code>	动态切换编程语言	<code>language</code> : 语言标识（如 <code>'python'</code> 、 <code>'json'</code> ）
<code>set_theme(theme: str)</code>	动态切换主题	<code>theme</code> : 主题名称（如 <code>'monokai'</code> 、 <code>'github'</code> ）
<code>bind_value(target)</code>	双向绑定编辑器内容 到目标组件（如文本 框、标签）	<code>target</code> : 目标组件； <code>target_name</code> : 目标属性名（默认 <code>value</code> ）
<code>bind_visibility(target)</code>	双向绑定组件可见性 到目标组件	<code>target</code> : 目标组件； <code>target_name</code> : 目标属性名（默认 <code>visible</code> ）
<code>on(event_name: str, callback: Callable)</code>	绑定 CodeMirror 原生事件	<code>event_name</code> : 事件名（如 <code>'focus'</code> 、 <code>'blur'</code> 、 <code>'cursorActivity'</code> ）； <code>callback</code> : 回调函数
<code>update()</code>	强制更新组件状态到 客户端	无参数
<code>delete()</code>	删除组件及所有子元 素	无参数

3. 支持的核心事件（CodeMirror 原生）

事件名	说明	回调参数
<code>change</code>	编辑器内容变化时触发（与 <code>on_change</code> 参数功能一致）	事件对象 <code>e</code> ，含 <code>value</code> 属性（当前内容）
<code>focus</code>	编辑器获得焦点时触发	事件对象（含 <code>view</code> 属性，CodeMirror 视图实例）

事件名	说明	回调参数
<code>blur</code>	编辑器失去焦点时触发	事件对象 (含 <code>view</code> 属性)
<code>cursorActivity</code>	光标移动或选择范围变化时触发	事件对象 (含 <code>view</code> 属性)
<code>keydown</code>	按下键盘按键时触发	事件对象 (含 <code>key</code> 属性, 按键标识)

六、注意事项

- 语言 / 主题兼容性:** `language` 和 `theme` 参数需为 CodeMirror 6 支持的标识, 不可随意自定义 (完整支持列表见 [CodeMirror 6 官方文档](#))。
- 性能优化:** 编辑超大文件 (1000+ 行) 时, 建议关闭 `linewrapping`、`foldGutter` 等功能, 减少渲染压力; 高频操作 (如实时保存) 需通过 `throttle` 控制回调频率。
- 第三方插件集成:** CodeMirror 6 支持插件扩展 (如 LSP 语法检查、代码片段), 需通过 NiceGUI 的 `ui.add_script()` 注入插件脚本, 再通过 `options` 配置启用 (需前端开发基础)。
- 样式冲突处理:** 自定义主题时, 建议使用 `!important` 覆盖默认样式, 避免与 Tailwind/Quasar 全局样式冲突。
- 版本兼容性:**
 - NiceGUI 2.0.0+ 支持 `ui.codemirror` 基础功能;
 - 2.16.0+ 支持 `html_id` 属性;
 - 3.0.0+ 支持更多 CodeMirror 6 原生配置 (如 `extraKeys` 快捷键);
 - 若需使用最新 CodeMirror 特性, 需确保 NiceGUI 框架为最新版本。
- 换行符处理:** 不同系统换行符 (`\n`/`\r\n`) 需统一, 建议存储时使用 `\n`, 避免编辑器显示异常。

七、应用场景

- 代码编辑工具: 如在线代码编辑器、Python 脚本编辑器、配置文件编辑器 (JSON/YAML/SQL)。
- 技术文档演示: 在文档中嵌入可编辑代码块, 支持实时修改和运行 (需结合后端执行逻辑)。
- Admin 系统配置: 允许管理员通过编辑器直接修改系统配置文件 (如 `config.py`、`app.yaml`)。
- 教育场景: 学生在线编写代码、提交作业, 教师在线批改 (支持语法高亮和格式化)。
- 代码分享平台: 用户分享代码片段, 支持语法高亮、主题切换、复制功能。

`ui.codemirror` 凭借 CodeMirror 6 的强大生态和 NiceGUI 的简洁 API, 成为 Python 后端开发者快速实现专业代码编辑功能的首选组件, 无需深入前端开发即可获得接近 VS Code 的编辑体验, 适配各类开发相关场景。